

```
"""
ARKHEIA-CPS – RTCSA Housing Service
Layer 3: Real-Time Contract Safety Analysis
for HOUSING contracts.
Orchestrates OIA-PEM Housing → weighted
risk score → alerts.
"""

from __future__ import annotations
from dataclasses import dataclass, field
from decimal import Decimal
from datetime import datetime
from typing import List

from sqlalchemy.orm import Session

from app.models.common import Contract,
AnalysisResult, VerticalType
from app.models.housing import
HousingContractDetails
from app.schemas.housing import
HousingContractIn
from app.schemas.common import Alert,
AlertSeverity, PEMEvaluation
from app.services.oia_pem_housing import
OIAPEMHousingService
from app.utils.explanations import
generate_housing_explanations
from app.utils.statutes import
get_housing_statutes
from app.utils.scoring import
```

```
compute_weighted_score, score_to_risk_level
```

```
# — Dimension weights
```

```
WEIGHTS = {  
    "affordability": 0.25,  
    "fee": 0.20,  
    "lease_term": 0.25,  
    "habitability": 0.15,  
    "eviction_safety": 0.15,  
}
```

```
@dataclass  
class RTCSAHousingResult:  
    overall_risk_score:  
        Decimal = Decimal("0")  
    risk_level:  
        str = "GREEN"  
    dimension_scores:  
        dict =  
        field(default_factory=dict)  
    alerts:  
        List[Alert] =  
        field(default_factory=list)  
    pem_evaluations:  
        List[PEMEvaluation] =  
        field(default_factory=list)  
    explanations:  
        List[str] =  
        field(default_factory=list)  
    statutes:
```

```

List[dict]          =
field(default_factory=list)
    total_movein_cost:
Decimal              = Decimal("0")
    total_monthly_recurring:
Decimal              = Decimal("0")
    illegal_clauses:
List[str]            =
field(default_factory=list)
    predatory_flags:
List[str]            =
field(default_factory=list)

class RTCSAHousingService:

    def __init__(self, db: Session):
        self.db = db
        self.pem = OIAPEMHousingService()

    def analyze(self, contract: Contract,
details: HousingContractDetails,
                payload: HousingContractIn)
-> RTCSAHousingResult:

        result = RTCSAHousingResult()

        # — Step 1: OIA-PEM evaluation


---


        pem_result =
self.pem.evaluate(payload)
        result.pem_evaluations =
pem_result.evaluations
        result.illegal_clauses =

```

```

pem_result.illegal_clauses
    result.predatory_flags =
pem_result.predatory_flags
    flags = pem_result.dimension_flags

# — Step 2: Compute summary cost
figures _____
    rent = payload.base_monthly_rent
    movein =
payload.security_deposit_amount
    if payload.first_month_rent_due:
        movein += rent
    if payload.last_month_rent_due:
        movein += rent
    movein +=
payload.application_fee_amount +
payload.admin_fee_amount
    movein +=
sum(Decimal(str(f.amount)) for f in
payload.other_upfront_fees)
    result.total_movein_cost = movein

    recurring = sum(
        Decimal(str(f.amount)) for f in
payload.recurring_fees
        if f.mandatory and f.frequency
== "monthly"
    )
    result.total_monthly_recurring =
recurring

# — Step 3: Dimension scoring
_____

dim = {}

```

```
        dim["affordability"] = (  
            85.0 if  
flags.get("affordability_fail") else  
            35.0 if  
flags.get("affordability_warn") else 5.0  
        )  
        dim["fee"] = (  
            80.0 if flags.get("fee_fail")  
else  
            30.0 if flags.get("fee_warn")  
else 5.0  
        )  
        dim["lease_term"] = (  
            75.0 if  
flags.get("lease_term_fail") else  
            30.0 if  
flags.get("lease_term_warn") else 5.0  
        )  
        dim["habitability"] = (  
            90.0 if  
flags.get("has_illegal_clauses") else  
            50.0 if  
flags.get("habitability_fail") else 5.0  
        )  
        dim["eviction_safety"] = (  
            80.0 if  
flags.get("eviction_fail") else  
            30.0 if  
flags.get("eviction_warn") else 5.0  
        )  
  
        result.dimension_scores = dim
```

```
# — Step 4: Overall score
```

```
    overall =  
    compute_weighted_score(dim, WEIGHTS)  
    result.overall_risk_score =  
    Decimal(str(overall))  
    result.risk_level =  
    score_to_risk_level(overall)
```

```
# — Step 5: Alerts from PEM
```

```
    for ev in pem_result.evaluations:  
        if ev.status in ("FAIL",  
"WARN"):  
            sev = AlertSeverity.HIGH if  
ev.status == "FAIL" else  
AlertSeverity.MODERATE  
            result.alerts.append(Alert(  
  
code=f"HOUSING_{ev.pattern_key[:25]}",  
            severity=sev,  
            message=ev.explanation  
or f"{ev.pattern_family} issue detected.",  
            ))  
  
    # Illegal clause alerts  
    for clause in  
pem_result.illegal_clauses:  
        result.alerts.append(Alert(  
            code="ILLEGAL_CLAUSE",  
  
severity=AlertSeverity.EXTREME,  
            message=(  
                f"Illegal clause
```

```

detected: '{clause}'. This clause may be "
            f"unenforceable under
tenant protection law and may indicate "
            f"a predatory
landlord."
        ),
    ))

    # Sort: EXTREME first
    severity_order = {"extreme": 0,
"high": 1, "moderate": 2, "low": 3}
    result.alerts.sort(key=lambda a:
severity_order.get(a.severity.value, 4))

    # — Step 6: Explanations
    

---


    result.explanations =
generate_housing_explanations(
        alerts=result.alerts,
        payload=payload,

illegal_clauses=pem_result.illegal_clauses,

total_movein=result.total_movein_cost,
    )

    # — Step 7: Statutes
    

---


    result.statutes =
get_housing_statutes(payload.jurisdiction_s
tate)

    # — Step 8: Persist
    

---



```

```

        self._persist(contract, result)

        return result

    def _persist(self, contract: Contract,
result: RTCSAHousingResult):
        dim = result.dimension_scores
        analysis = AnalysisResult(
            contract_id=contract.id,
            vertical=VerticalType.HOUSING,

overall_risk_score=result.overall_risk_score,
            risk_level=result.risk_level,

affordability_score=Decimal(str(dim.get("affordability", 0))),

fee_score=Decimal(str(dim.get("fee", 0))),

lease_term_score=Decimal(str(dim.get("lease_term", 0))),

habitability_score=Decimal(str(dim.get("habitability", 0))),

eviction_safety_score=Decimal(str(dim.get("eviction_safety", 0))),
            triggered_alerts_json=[
                {
                    "code": a.code,
"severity": a.severity.value,
                    "message": a.message,
                }
            ]

```



```

        for a in result.alerts
    ],
    statutes_json=result.statutes,

    explanations_json=result.explanations,
    pem_evaluations_json=[
        {
            "family":
e.pattern_family, "key": e.pattern_key,
            "status": e.status,
            "explanation": e.explanation,
            "actual":
str(e.actual_value) if e.actual_value else
None,
        }
        for e in
result.pem_evaluations
    ],
    created_at=datetime.utcnow(),
)
contract.analysis_timestamp =
datetime.utcnow()
self.db.add(analysis)
self.db.flush()

```